

From: James Stanier from The Engineering Manager theengineeringmanager@substack.com
Subject: Just build the tools yourself
Date: 20 February 2026 at 18:31
To: wvanheemstra@icloud.com



Forwarded this email? [Subscribe here](#) for more

Just build the tools yourself

You can have anything you want with so little effort.

JAMES STANIER

FEB 20 • [PREVIEW](#)



READ IN APP ↗

Welcome to February's subscribers edition, and thanks as always for your support. This one's a bit different: less theory, more show and tell.

Here's why: AI has made building simple tools almost as easy as building a spreadsheet, and I'm going to walk you through some I've built over recent months to solve common problems without needing to buy specialist software.

By the end, I hope that you'll feel motivated to get your hands dirty and close to the details and solve real problems (and it's fun!).

Think about how internal tooling used to work. You had two options: convince engineering to build it for you, which meant competing for roadmap space against customer-facing features, or settle for off-the-shelf SaaS that didn't quite fit your workflow and required convincing the company to pay for it.

Good internal tooling was a luxury reserved for the biggest tech companies; the ones who could afford dedicated teams to build custom software for management tasks. For everyone else, they made do with spreadsheets and workarounds.

Here's what we'll cover:

- Why the landscape has shifted and what that means for managers who used to code.
- Four tools I've built recently, from a bragdoc generator to a planning tool that embodies the [single prioritised list](#) philosophy from this month's free article.
- Some ideas for cool tools you could build.
- The practicalities: what this requires, where it breaks down, and when a tool has served its purpose.

If you'd like to dig deeper into related themes, here are some articles from the archive:

- [Being in the details](#) explores why staying close to what your teams are building matters more than ever.
- [Use it or lose it](#) covers the "minimum effective dose" of coding for managers who want to keep their skills sharp.
- [Should managers still code?](#) tackles the fundamental question that this article builds upon.

The Engineering Manager is a reader-supported publication. To receive new posts and support my work, consider becoming a free or paid subscriber.

[Upgrade to paid](#)

The new economics of tooling

The good news is that internal tooling is no longer a luxury reserved for large companies.

AI has democratised the ability to build custom tools, especially if you are time poor and a little rusty. The economics have fundamentally changed: if you have some technical background, and even if it's been years since you wrote code professionally, you can now build a working tool in an afternoon.

The capabilities of coding assistants at the time of writing are good enough to solve countless common problems, even if what you need is something you'd have considered paying for just a few years ago.

This isn't about you as a leader pivoting back to individual contributor again, or putting yourself on the critical path for shipping features. It's about removing the friction between "I wish I had a tool that did X" and actually having it. The problems worth solving this way are the ones too niche for anyone else to build for you: your specific workflow, your team's specific needs, the visibility gap that only matters in your context.

So, let me show you what I mean. Recently, I've built a handful of tools that have become part of how I work. Few of them took more than an afternoon. All of them solve problems that would have sat unsolved before.

And the liberating thing, other than having useful tools that solve real problems, is that knowing they don't need to be production-grade makes the process creative, fun, and something that just feels really good.

Cool things I've built

Bragdoc generator

I mentioned this one briefly in [Use it or lose it](#), but it's worth expanding on here. Here's the context: [performance review](#) season is tedious: you sit down to write your self-assessment and realise you've forgotten half of what you actually did. The evidence is scattered across Linear tickets, GitHub PRs, Slack threads, and your increasingly unreliable memory.

The concept of a "brag document" comes from [Julia Evans](#): a running record of your accomplishments that you update throughout the year so you're not scrambling to remember everything at review time. It's a brilliant idea, but I never managed to keep one up to date. So I built a tool to generate one for me instead.

My workplace uses Linear for everything, and I also track my personal work todos in it, which makes it a comprehensive record of my work. Given that it's

a system of truth, it follows that the actions I take in Linear represent my main achievements for the week.

The **bragdoc generator** pulls issue descriptions and comments from Linear for the past week, then uses a local LLM via Ollama to summarise what I've been working on. I run it weekly to generate a document that tells me what I've been up to, which I can then reflect on, use to plan the next week, and revisit at performance review time. It took about an hour to build with Claude Code.

It's not perfect, but it solves my problem well enough, and that's the point. Perfect is the enemy of done. If you're interested in giving it a go, you can fork it yourself, or repoint it to other tools if you don't use Linear.

The planning tool

This month's free article introduced **the single prioritised list**: one ordered list of what matters most, rather than the usual mess of backlogs, roadmaps, and priority tiers scattered across different tools and documents. We wanted to use it to facilitate planning discussions at a high level for 2026, so I built something.

The requirements were more ambitious than the bragdoc:

- A list of projects with their estimates and resourcing needs as submitted by the teams, along with metadata like home team and required skills: backend, frontend, QA.
- A list of all staff with their skill sets and home teams.
- The ability to drag projects into priority order to create the single prioritised list.
- Manual allocation by dragging people onto projects.
- Auto-allocation that considers priority, available skills, and a preference for keeping engineers in their home teams.
- A roadmap view showing what gets fully staffed, what's partially resourced, and what gets nothing at all.

- Import and export from CSV so we could pull data from existing spreadsheets.

Here's the first screenshot of what it looks like. All staff and projects are fictional...

Keep reading with a 7-day free trial

Subscribe to ***The Engineering Manager*** to keep reading this post and get 7 days of free access to the full post archives.

Start trial

A subscription gets you:

- ✓ Monthly paid subscriber posts, which go deeper and more personal than my public writing.
- ✓ Ability to chat directly to me and influence the future of the newsletter
- ✓ Feel warm and fuzzy inside



LIKE



COMMENT



RESTACK

© 2026 James Stanier

548 Market Street PMB 72296, San Francisco, CA 94104

[Unsubscribe](#)



Start writing